IKT215 Exercise 7:
Function Approximation Classification (Part I)

March 12th, 2026

Exercise sessions, while not mandatory, are an integral part of your learning experience and exam preparation. Some exam questions will be based on tasks covered during these sessions. Although you have the flexibility to complete exercises at home and ask questions later (preferably within one week), I strongly recommend attending the sessions in person for immediate support and guidance. This allows you to benefit from direct interaction and real-time problem-solving. I respond promptly to emails (manuel.s.mathew@uia.no).

Question 1: Rosenblatt's Perceptron

Dataset: synthetic dataset generation

1. Create 2D synthetic data with 2 classes using Gaussian distributions:
    a. Class 0: Mean (0, 0), Std dev 0.8
    b. Class 1: Mean (1, 1), Std dev 0.8
    c. Generate 200 samples (100 per class)
    d. Split into 70% train:30% test
2. Plot training data with class colors with axis labels and legend.
3. Implement perceptron as a class with these methods:
    a. `__init__`: Initialize weights (include bias term)
    b. `predict`: Step function activation
    c. `fit`: Training loop with weight updates
4. Training & Evaluation
    a. Train for 50 epochs with learning rate $\eta = 0.1$
    b. Plot final decision boundary over training data
    c. Calculate test accuracy
    d. Plot test set predictions with decision boundary
5. Explore limitations:
    a. Modify data generation (increase std dev to 1.2)
    b. Observe impact on decision boundary & accuracy
    c. Experiment with different learning rates (0.01 vs 0.5)

Question 2: Linear SVM vs Non-Linear SVM

1. Dataset Generation
   We'll use `make_moons` dataset from `sklearn` with added noise for non-linear separation:

   ```
   from sklearn.datasets import make_moons
   # Generate 1000 samples with 30% noise
   X, y = make_moons(n_samples=1000, noise=0.3, random_state=42)
   ```
2. Linear SVM Implementation
   a. Train a linear SVM with `C=0.1`
   b. Visualize decision boundary
   c. Calculate training/test accuracy
   d. Repeat with different C values and compare results
3. Non-Linear SVM with RBF Kernel
   a. Implement RBF kernel SVM with `C=1` and `gamma='scale'`
   b. Visualize decision boundary
   c. Compare accuracy with linear models
4. C Parameter Analysis
   a. Create models with `C=[0.1, 1, 100]` (keep gamma fixed)
   b. Plot decision boundaries side-by-side
   c. Explore train/test accuracies for each C value
   d. Train high-C model (C=1000) on reduced dataset (first 100 samples)
      i. Show perfect training accuracy but poor test performance
      ii. Visualize complex boundary